

SQS vs SNS vs Kafka vs RabbitMQ

Fundamental differences, use cases, consumer scaling and visibility timeout

The One-Line Mental Model

SQS -- a queue. One producer drops a message; one consumer picks it up.

SNS -- a megaphone. One publisher broadcasts; many subscribers receive it simultaneously.

Kafka -- a distributed log. Messages are stored durably and any consumer can replay from any point in time.

RabbitMQ -- a smart broker. Messages are routed with rich rules; the broker does the heavy lifting; consumed messages are deleted.

SQS -- Point-to-Point Queue

The fundamental guarantee: **each message is processed by exactly one consumer**. Once consumed it is gone.

SQS decouples a producer from a consumer so they do not need to be running at the same time.

```
[SQS Queue]
|
+---> Consumer Instance 1 (picks up message A)
+---> Consumer Instance 2 (picks up message B)
+---> Consumer Instance 3 (picks up message C)
```

Use Cases

- **Order processing** -- place an order, drop it in a queue, payment service processes it asynchronously. No duplicate charges.
- **Background jobs** -- image resize, PDF generation, email sending. Workers scale up/down independently.
- **Rate limiting** -- a queue naturally absorbs traffic spikes. A burst of 10,000 requests does not overwhelm a downstream service handling 100/sec.
- **Dead letter queues** -- failed messages after N retries move to a DLQ automatically.

What SQS cannot do

Fan-out. If two services both need to react to the same event, the message goes to only one consumer. Combine with SNS for fan-out.

SNS -- Publish / Subscribe Fan-out

The fundamental guarantee: **one message reaches all subscribers simultaneously**. SNS does not store messages -- it fires and forgets.

```
Payment Completed Event
|
SNS Topic
/ | \ \
Email SMS Slack SQS Queue (-> Warehouse Service)
```

Use Cases

- **Event fan-out** -- a payment completes; simultaneously trigger an email receipt, update inventory, notify the warehouse, and log to analytics.
- **Mobile push notifications** -- SNS natively integrates with APNS (iOS) and FCM (Android) to broadcast to millions of devices.
- **Cross-service event propagation** -- microservices publish domain events to SNS; each downstream service subscribes with its own SQS queue.

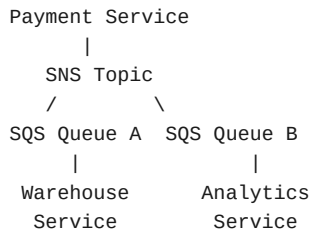
- **Alerts** -- CloudWatch alarms publish to SNS; ops team gets paged via SMS/email simultaneously.

What SNS cannot do

Guarantee ordering, store messages for replay, or handle a subscriber that is temporarily down. If no subscriber catches the message at that moment, it is lost.

The SNS + SQS Combo (Most Common AWS Pattern)

SNS handles fan-out. SQS handles durability and per-service consumption rate. Each service gets its own queue and processes at its own pace.



Kafka -- Distributed Commit Log

Kafka is fundamentally different: **messages are not deleted after consumption**. They are written to a durable ordered log and retained for a configured period. Any consumer can read from any offset -- past, present, or future.

```

Producer -> [Topic: orders] -> Consumer Group A (warehouse) <- offset 450
                                -> Consumer Group B (analytics)   <- offset 200
                                -> Consumer Group C (new service) <- offset 0 (replay)
  
```

Use Cases

- **Event sourcing** -- your entire order history is the Kafka log. A new microservice can replay all events from day one to build its own state.
- **Real-time stream processing** -- feed Kafka into Flink or Spark Streaming to compute rolling aggregates, fraud detection signals, or live dashboards.
- **Audit log / compliance** -- immutable, ordered record of everything that happened. Append-only log with configurable retention.
- **CDC (Change Data Capture)** -- Debezium captures every DB row change into Kafka; downstream systems (search index, cache, warehouse) all consume independently.
- **High throughput** -- handles millions of messages/sec via sequential disk writes.

What Kafka requires

Operational overhead. It is a cluster you run and manage (or pay for via Confluent/MSK). SQS and SNS are fully managed with zero ops.

RabbitMQ -- Traditional Message Broker

Broker-centric vs Consumer-centric

RabbitMQ is broker-centric. The broker holds routing logic, tracks delivery, and deletes messages after acknowledgement. The broker does the heavy lifting.

Kafka is consumer-centric. The broker just appends to a log. Consumers track their own offset. The consumer does the heavy lifting.

```

RabbitMQ:
Producer -> [Smart Broker] -> routes -> Queue A -> Consumer 1
                                -> routes -> Queue B -> Consumer 2
  
```

Kafka:

```

Producer -> [Dumb Log: msg0, msg1, msg2, msg3...]
           Consumer A reads at offset 6
           Consumer B reads at offset 3 <- different speed, same log
           Consumer C reads at offset 0 <- replaying from start

```

RabbitMQ's Superpower: Flexible Routing

RabbitMQ has a sophisticated exchange/binding model. You express complex routing entirely in the broker -- producers and consumers do not need to know about each other.

```

Producer publishes: { type: 'payment.failed', region: 'UK' }

```

```

Direct Exchange  -> exact match routing key  -> payment-failed-queue
Topic Exchange   -> 'payment.*'              -> all-payments-queue
                  -> '*.failed'              -> all-failures-queue
Fanout Exchange  -> ignores key               -> every bound queue
Headers Exchange -> matches on msg headers   -> region-specific queues

```

Kafka has none of this. You route by writing to different topics. Filtering happens in consumer code, not the broker.

Kafka's Superpower: Retention and Replay

RabbitMQ deletes a message the moment it is acknowledged -- it is gone. Kafka retains messages regardless of consumption, enabling replay, new consumer bootstrapping, and multiple independent consumer groups on the same stream. RabbitMQ fundamentally cannot do any of these.

RabbitMQ Use Cases

- **Task queues with complex routing** -- a logistics platform routing shipment events to UK, EU, or fragile-handling queues based on content. Topic exchange handles this elegantly in the broker.

```

shipment.uk.fragile -> UK queue + fragile-handling queue
shipment.eu.standard -> EU queue only

```

- **RPC (request/reply) patterns** -- a microservice calls another and waits for a response. RabbitMQ has native correlation ID and reply-to queue support.

```

Service A -> [request queue] -> Service B processes -> [reply queue] -> Service A

```

- **Priority queues** -- urgent jobs jumping ahead of normal ones. RabbitMQ supports message priority natively. Kafka has no concept of priority.
- **Per-message TTL and expiry** -- a notification relevant for only 60 seconds. RabbitMQ can expire individual messages. Kafka retention is topic-wide only.

RabbitMQ vs Kafka

	RabbitMQ	Kafka
Architecture	Broker-centric (smart broker)	Consumer-centric (dumb log)
Routing	Rich: direct, topic, fanout, headers	By topic only; filter in consumer
Retention	Deleted after ack	Configurable (days to forever)
Replay	No	Yes -- from any offset
Ordering	Within queue + single consumer	Within partition (by key)
Priority	Yes -- native	No
Per-message TTL	Yes	No (topic-level only)
Throughput	Tens of thousands/sec	Millions/sec
Latency	Sub-millisecond	Single-digit ms

	RabbitMQ	Kafka
Ops overhead	Medium	High
Best for	Complex routing, RPC, task queues	Streams, event sourcing, CDC

***When they overlap:** Both handle async messaging between services. Choose RabbitMQ when routing logic is complex and messages are transient. Choose Kafka when you need replay, multiple independent consumers, or sustained high throughput.*

Consumer Scaling -- SQS vs Kafka

SQS: Competing Consumers (No Ceiling)

SQS scales by running multiple instances. SQS coordinates via visibility timeout. No groups, no partition assignments, no rebalancing. 500 Lambda functions can all consume from the same queue simultaneously.

Kafka: Consumer Groups (Partition Ceiling)

If your topic has 10 partitions, you can have at most 10 active consumers in a group. An 11th consumer sits idle until the partition count is increased.

```
Kafka Topic: 3 partitions, 5 consumer instances
Instance 1 -> Partition 0 (active)
Instance 2 -> Partition 1 (active)
Instance 3 -> Partition 2 (active)
Instance 4 -> idle
Instance 5 -> idle
```

Ordering Trade-off in SQS

SQS distributes to whoever polls first -- ordering across consumers is not guaranteed. Use **SQS FIFO with Message Group ID** to process all messages for a given key in sequence. This is the same concept as Kafka partition keys.

```
order_id=123 -> always one consumer, in order
order_id=456 -> different consumer, also in order
```

Visibility Timeout -- What Happens When It Lapses

SQS does not know whether a consumer finished the work or crashed mid-way. It only knows the message was not deleted within the timeout window, so it makes the message visible again.

```
Consumer 1 picks up message A
Visibility timeout starts (30s)
Consumer 1 slow / crashes / hangs
Timeout expires -- message becomes VISIBLE again
Consumer 2 picks up message A <- duplicate processing
```

At-least-once delivery. Your consumers MUST be idempotent.

Making Consumers Idempotent

```
# WRONG -- not idempotent
def process(order_id):
    db.execute('INSERT INTO charges VALUES (?)', order_id)
    # Runs twice = double charge

# RIGHT -- idempotent
def process(order_id):
    db.execute("""
        INSERT INTO charges VALUES (?)
        ON CONFLICT (order_id) DO NOTHING
    """, order_id)
```

How to Prevent Duplicate Processing

- **Set timeout longer than worst-case processing time.** Job takes 2 min? Set 5 min.
- **Extend mid-processing** via `change_message_visibility()` as a heartbeat.
- **FIFO queues** -- exactly-once delivery within a 5-minute deduplication window.
- **Track processed IDs in your DB** -- application-level dedup using the SQS message ID.

```
def process(message_id, order_id):
    if db.exists('SELECT 1 FROM processed WHERE id = ?', message_id):
        return # already done, skip
    # ... do work ...
    db.execute('INSERT INTO processed VALUES (?)', message_id)
    sqs.delete_message(ReceiptHandle=receipt_handle)
```

Dead Letter Queue Safety Net

Set `maxReceiveCount`. After N retries, SQS automatically moves the message to a DLQ so a poison pill message never loops forever.

Kafka Comparison

In Kafka a consumer commits its offset only after successful processing. If it crashes, it resumes from the last committed offset. The window for duplicates is tighter and fully under your control.

Full Comparison -- All Four

	SQS	SNS	RabbitMQ	Kafka
Model	Queue	Pub/Sub	Broker routing	Distributed log
Consumers	One per message	All subscribers	One per message	Multiple groups
Retention	Until consumed	None	Until acked	Configurable
Replay	No	No	No	Yes
Ordering	FIFO only	No	Per queue	Per partition
Routing	None	Topic/filter	Rich exchanges	By topic only
Priority	No	No	Yes	No
Per-msg TTL	No	No	Yes	No
Throughput	High	High	High	Extreme
Ops overhead	Zero	Zero	Medium	High
Best for	Task queues	Fan-out alerts	Complex routing, RPC	Streams, audit, CDC

How to Choose

- **Use SQS** when one service hands off work to another and you need durability with one-consumer-per-message semantics.
- **Use SNS** when one event needs to trigger multiple independent services simultaneously.
- **Use SNS + SQS together** (the standard AWS pattern) for fan-out with per-service durable queuing.
- **Use RabbitMQ** when routing logic is complex (topic exchanges, headers, priority, per-message TTL) or you need native RPC request/reply.

- **Use Kafka** when you need replay, long-term retention, stream processing, or throughput the others cannot match -- and you accept the operational cost.

***The golden rule:** SQS and SNS give you operational simplicity at the cost of replay and consumer control. RabbitMQ adds routing power with moderate ops. Kafka is maximum power at maximum operational cost. Start with SQS/SNS on AWS; consider RabbitMQ when routing gets complex; graduate to Kafka when replay, multiple independent consumers, or extreme throughput becomes a hard requirement.*